```
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

/** Some constants for ESP8266 **/
#define ESP8266_STATION         0x01
#define ESP8266_SOFTAP          0x02
#define ESP8266_BOTH            0x03

#define ESP8266_TCP             1
#define ESP8266_UDP             0
#define ESP8266_TRANS_PASS      1
#define ESP8266_TRANS_NOR       0

#define ESP8266_OK              1
#define ESP8266_READY           2
#define ESP8266_FAIL            3
#define ESP8266_NOCHANGE        4
#define ESP8266_LINKED          5
#define ESP8266_UNLINK          6
#define ESP8266_CONNECT         7

char TransmitData, ReceiveData;
bit transmit_flag, receive_flag;
//==============================================================================
void interrupt(void)
{
   if((RCIE_bit == 1) && (RCIF_bit == 1))
   {
      RCIF_bit = 0;                 // Clear interrupt bit
      ReceiveData = UART1_Read();   // Doc du lieu tu UART
      receive_flag = 1;
   }
}
//==============================================================================
//**Function to send one byte of date to UART**//
void _esp8266_putch(char bt)
{
   while(!TXIF_bit);  // hold the program till TX buffer is free
   TXREG = bt; //Load the transmitter buffer with the received value
}
//==============================================================================
//**Function to get one byte of date from UART**//
char _esp8266_getch()
{
   if(OERR_bit)      // check for Error
   {
      CREN_bit = 0;  //If error -> Reset
      CREN_bit = 1;  //If error -> Reset
   }
   while(!RCIF_bit); // hold the program till RX buffer is free
   return RCREG;      //receive the value and send it to main function
}
```

```
//============================================================================
//**Function to convert string to byte**//
void ESP8266_send_string(char* st_pt)
{
   while(*st_pt) //if there is a char
   {
      _esp8266_putch(*st_pt++); //process it as a byte data
   }
}
//============================================================================
/**
 * Output a string to the ESP module.
 * This is a function for internal use only.
 * @param ptr A pointer to the string to send.
 */
void _esp8266_print(unsigned char *ptr) {
   while (*ptr != 0)
   {
      _esp8266_putch(*ptr++);
   }
}
//============================================================================
/**
 * Wait until we found a string on the input.
 * Careful: this will read everything until that string (even if it's never
 * found). You may lose important data.
 * @param string
 * @return the number of characters read
 */
inline uint16_t _esp8266_waitFor(unsigned char *string)
{
   unsigned char so_far = 0;
   unsigned char received;
   uint16_t counter = 0;
   do
   {
      received = _esp8266_getch();
      counter++;
      if (received == string[so_far])
      {
         so_far++;
      }
      else
      {
         so_far = 0;
      }
   }
   while (string[so_far] != 0);
   return counter;
}
//============================================================================
/**
 * Restart the module
 * This sends the `AT+RST` command to the ESP and waits until there is a
```

```
 * response "OK" and "ready".
 */
void esp8266_restart(void)
{
   _esp8266_print("AT+RST\r\n");           // Command
   _esp8266_waitFor("OK");                 // Wait for "OK"
   _esp8266_waitFor("ready");              // Wait for "ready"
}
//============================================================================
/**
 * Check if the module is started
 * This sends the `AT` command to the ESP and waits until it gets a response "OK".
 */
void esp8266_isStarted(void)
{
   _esp8266_print("AT\r\n");           // Command
   _esp8266_waitFor("OK");             // Wait for "OK"
}
//============================================================================
/**
 * Enable / disable command echoing.
 * Enabling this is useful for debugging: one could sniff the TX line from the
 * ESP8266 with his computer and thus receive both commands and responses.
 * This sends the ATE command to the ESP module and waits until it gets a response "OK".
 * @param echo whether to enable command echoing(1) or not(0)
 */
void esp8266_echoCmds(bool echo)
{
   _esp8266_print("ATE");           // Command
   if (echo)
   {
      _esp8266_putch('1');
   }
   else
   {
      _esp8266_putch('0');
   }
   _esp8266_print("\r\n");

   _esp8266_waitFor("OK");       // Wait for "OK"
}
//============================================================================
/**
 * Set the WiFi mode.
 * ESP8266_STATION : Station mode
 * ESP8266_SOFTAP : Access Point mode
 * ESP8266_BOTH : Station + Access Point mode
 * This sends the AT+CWMODE command to the ESP module and waits until it gets a response "OK".
 * @param mode an ORed bitmask of ESP8266_STATION, ESP8266_SOFTAP, ESP8266_BOTH
 */
void esp8266_mode(unsigned char mode)
{
   _esp8266_print("AT+CWMODE=");     // Command
   _esp8266_putch(mode + '0');
```

```
   _esp8266_print("\r\n");
   _esp8266_waitFor("OK");           // Wait for "OK"
}
//============================================================================
/**
 * Set the transmission mode.
 * ESP8266_TRANS_PASS : Passthrough Receiving Mode (Transparent receiving transmission
 * ESP8266_TRANS_NOR : Normal Transmission Mode
 * This sends the AT+CIPMODE command to the ESP module and waits until it gets a response "OK".
 * @param mode an ORed bitmask of ESP8266_TRANS_PASS, ESP8266_TRANS_NOR
 */
void esp8266_trans_mode(unsigned char mode)
{
   _esp8266_print("AT+CIPMODE=");     // Command
   _esp8266_putch(mode + '0');
   _esp8266_print("\r\n");
   _esp8266_waitFor("OK");           // Wait for "OK"
}
//============================================================================
/**
 * Connect to an access point.
 * This sends the AT+CWJAP command to the ESP module and waits until it gets a response "OK".
 * @param ssid The SSID to connect to
 * @param pass The password of the network
 */
void esp8266_connect(unsigned char* ssid, unsigned char* pass)
{
   _esp8266_print("AT+CWJAP=\"");      // Command
   _esp8266_print(ssid);
   _esp8266_print("\",\"");
   _esp8266_print(pass);
   _esp8266_print("\"\r\n");
   _esp8266_waitFor("OK");       // Wait for "OK"
}
//============================================================================
/**
 * Open a TCP or UDP connection.
 * This sends the AT+CIPSTART command to the ESP module and waits until it gets a response "OK".
 * @param protocol Either ESP8266_TCP or ESP8266_UDP
 * @param ip The IP or hostname to connect to; as a string
 * @param port The port to connect to
 */
unsigned char esp8266_start(unsigned char protocol, unsigned char* ip, unsigned int port)
{
   unsigned char port_str[5] = "\0\0\0\0";
   _esp8266_print("AT+CIPSTART=\"");
   if (protocol == ESP8266_TCP)
   {
      _esp8266_print("TCP");
   }
   else
   {
      _esp8266_print("UDP");
   }
```

```
   _esp8266_print("\",\"");
   _esp8266_print(ip);
   _esp8266_print("\",");

   sprintf(port_str, "%u", port);
   _esp8266_print(port_str);
   _esp8266_print("\r\n");

   _esp8266_waitFor("OK");       // Wait for "OK"
}
//============================================================================
// Send data (AT+CIPSEND)
/**
 * Send data over a connection.
 * This sends the AT+CIPSEND command to the ESP module and waits until it gets a response "OK".
 * @param data The data to send
 */
void esp8266_send(void)   // For TCP Client Single Connection UART-Wifi Passthrough
{
   _esp8266_print("AT+CIPSEND");
   _esp8266_print("\r\n");
   _esp8266_waitFor("OK");
   while (_esp8266_getch() != '>');
}
//============================================================================
/**
 * Read a string of data that is sent to the ESP8266 (for Single Connection as TCP Client).
 * This waits for a +IPD line from the module. If more bytes than the maximum
 * are received, the remaining bytes will be discarded.
 * @param store_in a pointer to a character array to store the data in
 */
void esp8266_receive(unsigned char* store_in)
{
   unsigned char length = 0;
   unsigned char i;
   unsigned char received;

   // Format: +IPD,<length>:<data>
   _esp8266_waitFor("+IPD,");            // Wait for: +IPD - Read data
   do
   {
      received = _esp8266_getch();       // Get bytes of Length
      if(received == ':') break;         // Stop when detect ":"
      length = length * 10 + (received - '0');    // Determine data length
   }
   while (received >= '0' && received <= '9');


   for (i = 0; i < length; i++)          // Get bytes of Data
   {
      store_in[i] = _esp8266_getch();
   }
}
//============================================================================
```

```
/**
 * Disconnect from the access point.
 * This sends the AT+CWQAP command to the ESP module.
 */
void esp8266_disconnect(void)
{
   _esp8266_print("AT+CWQAP\r\n");
   _esp8266_waitFor("OK");
}
//================================================================================
/**
 * Stop sending data.
 * This sends the +++ to the ESP module.
 */
void esp8266_stop_send(void)
{
   _esp8266_print("+++");
   delay_ms(2000);
}
//================================================================================
/**
 * Delete TCP connection
 * This sends the AT+CIPCLOSE command to the ESP module.
 */
void esp8266_del_TCP(void)
{
   _esp8266_print("AT+CIPCLOSE\r\n");
   _esp8266_waitFor("OK");
}
//================================================================================
void main(void)
{
// Khai bao

// Chuong trinh
   ADCON1 |= 0x0F;
   CMCON  |= 7;

   // Cau hinh Port B
   PORTB = 0x00; LATB = 0x00;
   TRISB0_bit = 1;

   // Cau hinh Port E
   PORTE = 0x00; LATE = 0x00;
   TRISE0_bit = 0;      // LED for Connected
   TRISE1_bit = 0;

   // Cau hinh Port C
   PORTC = 0x00; LATC = 0x00;
   TRISC0_bit = 0;      // Reset pin of ESP8266

   UART1_Init(115200);
   delay_ms(100);
```

```
   // Reset for ESP8266 module
   RC0_bit = 0; delay_ms(100); RC0_bit = 1;

   delay_ms(1000);   // For ESP8266 stable

   //=========================================================
   // Configure: Station, TCP Client Single connection UART-Wifi Passthrough
   esp8266_restart();                                    // Command: AT+RST - Restart module
   esp8266_echoCmds(0);                                  // Command: ATE0 - Enable(1)/Disable(0) echo
   esp8266_isStarted();                                  // Command: AT - Wait till the ESP send back
"OK"
   esp8266_mode(ESP8266_STATION);                        // Command: AT+CWMODE=1 - Put the module in
Station  mode
   //esp8266_connect("IOT_AI LAB","rC7YNUqM");           // Command: AT+CWJAP="ssid","pass" -
Connect to an access point
   //esp8266_start(ESP8266_TCP, "192.168.1.49", 8000);   // Command:
AT+CIPSTART="protocol","ip",port - Start a TCP network in that IP and port
   //esp8266_connect("Smart_Home","pqtri2002");
   //esp8266_start(ESP8266_TCP, "192.168.0.106", 8000);
   esp8266_connect("X705","12345");
   esp8266_start(ESP8266_TCP, "192.168.1.31", 8000);

   RE0_bit = 1;                                          // Warning - Connect to Server (LED Connect
ON)
   esp8266_trans_mode(ESP8266_TRANS_PASS);               // Command: AT+CIPMODE="mode" - Select
Passthrough Receiving Mode (Transparent Receiving Transmission)
   esp8266_send();                                       // Command: AT+CIPSEND - Send data
   //=========================================================

   RCIF_bit = 0;         // Clear interrupt bit
   PIE1.RCIE = 1;         // Enable the EUSART receive interrupt

   GIE_bit = 1;        // Enable global interrupt
   PEIE_bit = 1;       // Enable periphiral interrupts

   while(1)
   {
      /******* Button 1 ***************************/
      if (Button(&PORTB, 0, 10, 0))
      {
         while(Button(&PORTB, 0, 10, 0));
         TransmitData = 'S';
         UART1_Write(TransmitData);
      }

      if(receive_flag == 1)
      {
         receive_flag = 0;
         ReceiveData = UART1_Read();
         if (ReceiveData == '@')
         {
            LATE1_bit = 1;
            TransmitData = 'O';
            UART1_Write(TransmitData);
```

```
         }
         else if (ReceiveData == '$')
         {
            LATE1_bit = 0;
            TransmitData = 'F';
            UART1_Write(TransmitData);
         }
         else if (ReceiveData == 'Z')      // Ngat ket noi
         {
            GIE_bit = 0;                              // Disable global interrupt
            RE0_bit = 0;                              // Warning: Disconnect Server (LED Connect OFF)

            esp8266_stop_send();                      // Command: +++ - Stop sending data
            esp8266_trans_mode(ESP8266_TRANS_NOR);    // Command: AT+CIPMODE="mode" - Disable UART -
WiFi passthrough mode (transparent transmission)
            esp8266_del_TCP();                        // Command: AT+CIPCLOSE - Delete TCP connection
         }
      }
   }
}
//=============================================================================
```